Desenvolvimento Web 1

Prof. Diego Max

Aula 07:

CSS3 – Estilização Básica e Seletores.

O Papel do CSS no Desenvolvimento Web

CSS (Cascading Style Sheets) é a linguagem responsável pela apresentação visual de documentos HTML. Enquanto o HTML define a estrutura e o conteúdo, o CSS controla **cores**, **fontes**, **layouts**, **espaçamentos** e **animações**.

A separação entre conteúdo (HTML) e apresentação (CSS) permite maior flexibilidade, manutenção facilitada e reutilização de estilos em múltiplas páginas. É um dos três pilares fundamentais do desenvolvimento web moderno, junto com HTML e JavaScript.

CSS3: A Evolução da Estilização Web

CSS3 representa a **terceira geração do CSS**, introduzindo recursos revolucionários que expandiram as possibilidades do web design.

Novos módulos do CSS3:

Novos Seletores: Seletores mais poderosos e específicos.

Efeitos Visuais: Sombras, gradientes, bordas arredondadas e transformações 2D/3D.

Animações: Transições suaves e animações complexas sem JavaScript.

Layouts Responsivos: Flexbox e Grid para designs adaptativos e modernos.

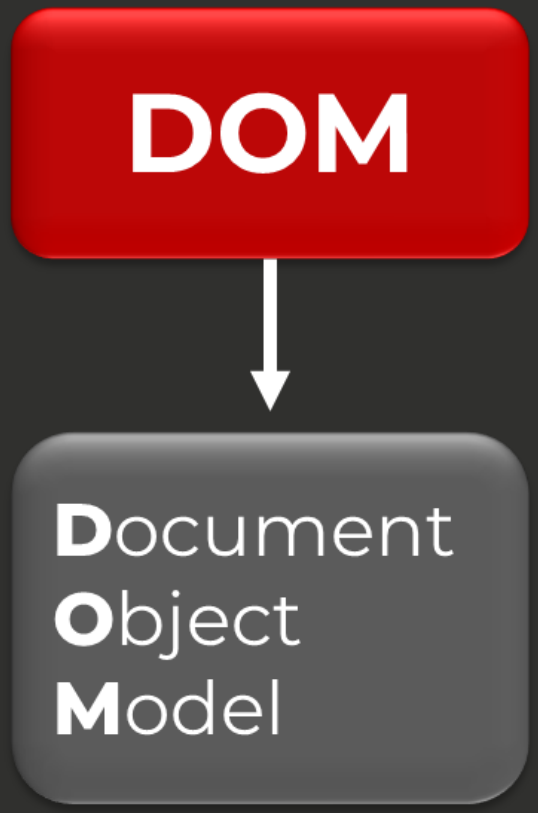
Conhecendo o DOM (Document Object Model)

Conhecer o **DOM (Document Object Model)** é importante antes de aprender CSS porque é ele que representa a estrutura da página em forma de árvore.

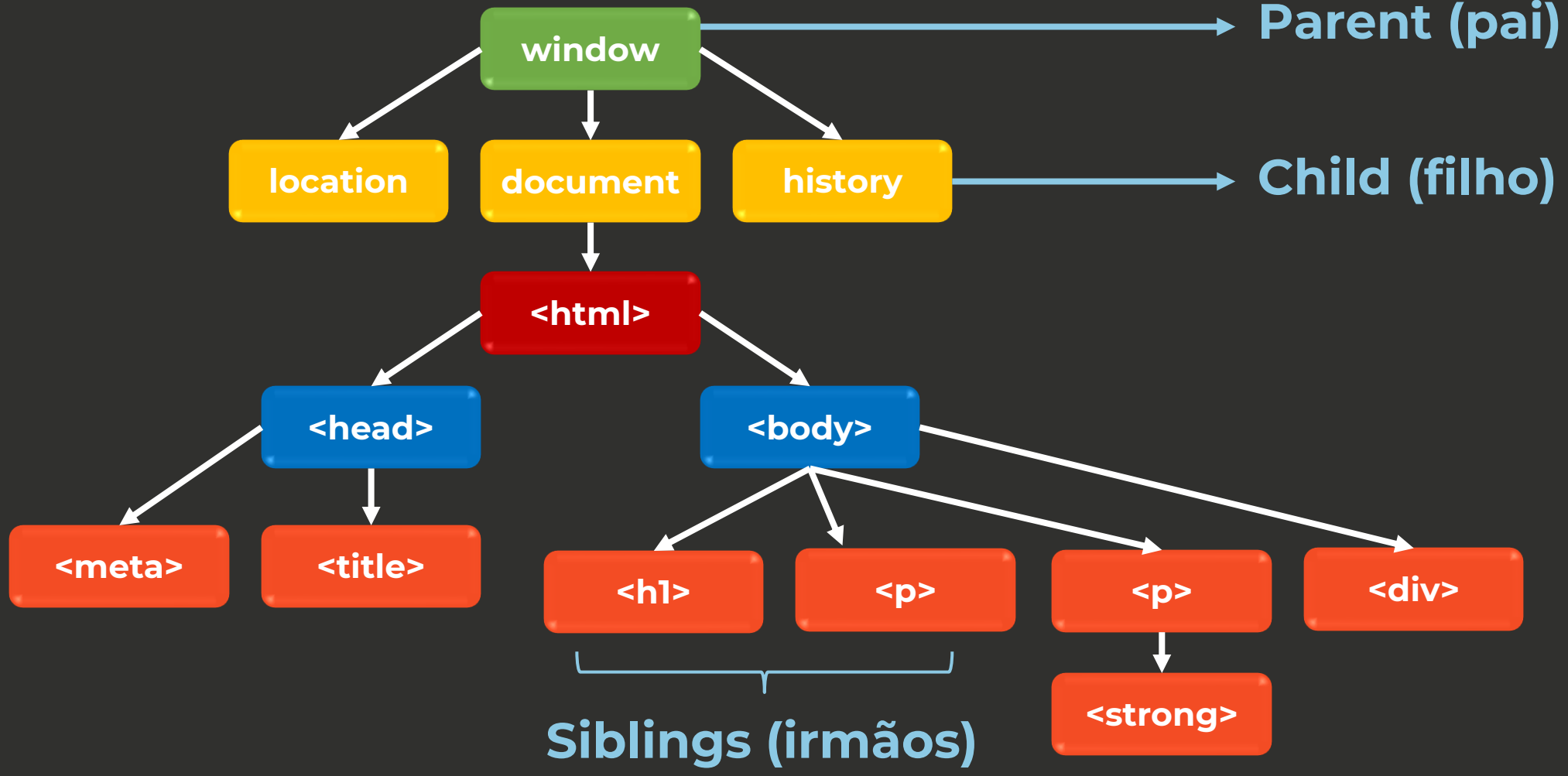
O CSS não estiliza o HTML “cru”, mas sim os elementos do DOM, usando seletores que dependem das relações entre nós (**pai, filho, irmãos**). Sem entender o DOM, fica difícil aplicar estilos corretamente, prever heranças, especificidade e o comportamento dos seletores.

O DOM é o **Modelo de Objetos para Documentos**. É um conjunto de **objetos** dentro do seu navegador que vai dar acesso aos componentes internos do website.

O DOM é criado pelo navegador assim que ele interpreta o HTML.



Árvore DOM


window
Parent (pai)
location
document
history
Child (filho)
<html>
<head>
<body>
<meta>
<title>
<h1>
<p>
<p>

Siblings (irmãos)

```html
<!DOCTYPE html>

<html lang="pt-br">

  <head>
      <meta charset="UTF-8">
      <meta http-equiv="X-UA-Compatible" content="IE=edge">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>Primeiros passos com DOM</title>
  </head>

  <body>
      <h1>Iniciando os estudos com DOM.</h1>
      <p>Aqui vai o resultado</p>
      <p>Aprendendo a usar o <strong>DOM</strong> em Javascript</p>
      <div>Clique em mim</div>
  </body>

</html>
```

Tópicos desta aula

- CSS Inline (inline CSS)
- CSS interno (internal CSS)
- CSS externo (external CSS)
- Seletores básicos
- Seletores avançados
- Pseudo classes e Pseudo elementos
- Especificidade
- Ferramentas do Desenvolvedor: Inspeção de CSS
- Boas Práticas e Armadilhas Comuns

Aula 07.1:

CSS Inline (inline CSS)

CSS Inline: Estilização Direta no Elemento

O **CSS inline** aplica estilos diretamente aos elementos HTML através do atributo **style**.
É o método mais direto, mas **menos recomendado** para **projetos maiores**.

Sintaxe Básica:

```html
<p style="color: #8CC9E4; font-size: 18px;">
    Texto estilizado
</p>
```

Texto estilizado

Atributos Comuns:

color – Define a cor do texto

font-size – Tamanho da fonte

background-color – Cor de fundo

margin e padding – Espaçamentos

CSS Inline: Exemplos Práticos

Exemplo 1: Estilização de Texto

```
<h1 style="color: #8CC9E4;
    font-family: Montserrat;
    text-align: center;">
    Título Principal
</h1>
```

Título Principal

Exemplo 2: Botão Estilizado

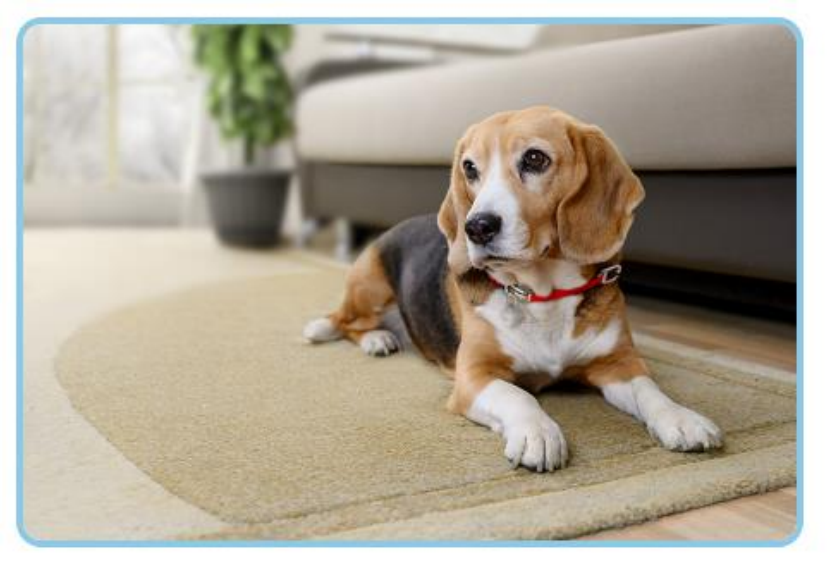
```
<button style="background-color: #8CC9E4;
    color: #30302E;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;">
    Clique Aqui
</button>
```

Clique Aqui

CSS Inline: Exemplos Práticos

Exemplo 3: Imagem com Borda

```
<img src="foto.jpg"
    style="border: 3px solid #8CC9E4;
    border-radius: 10px;
    width: 300px;">
```



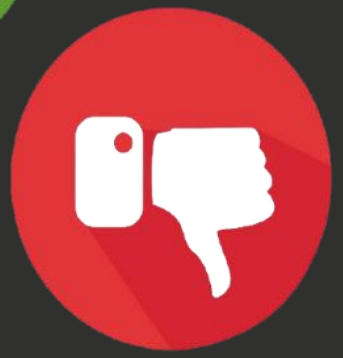
Vantagens e Desvantagens do CSS Inline

Vantagens:

- Aplicação rápida e imediata de estilos
- Alta especificidade – sobrescreve outros estilos facilmente
- Útil para testes e prototipagem rápida
- Ideal para estilos únicos e específicos
- Não requer arquivos externos

Desvantagens:

- Dificulta manutenção em projetos grandes
- Código HTML fica poluído e menos legível
- Impossível reutilizar estilos em múltiplos elementos
- Aumenta o tamanho dos arquivos HTML
- Viola o princípio de separação de responsabilidades



Aula 07.2:

CSS interno (internal CSS)

CSS Interno: Estilização no Documento

O CSS interno (ou incorporado) é definido dentro da seção **<head>** do documento HTML, usando a tag **<style>**. Este método permite aplicar estilos a múltiplos elementos na mesma página usando **seletores**.

Estrutura Básica:

```
<head>
    <style>
        seletor {
            propriedade: valor;
        }
    </style>
</head>
```

Exemplo completo:

```
<head>
    <style>
        h1 {
            color: #8CC9E4;
            font-family: Montserrat;
        }

        p {
            color: #FFFFFF;
            font-size: 18px;
        }
    </style>
</head>
```

Seletores Básicos no CSS Interno

Seletor de Elemento:

Aplica estilo a todas as **tags** de um tipo específico

```
p {
    color: white;
}
```

Seletor de Classe:

Usa o atributo **class** para *targeting* específico

```
.destaque {
    background: #8CC9E4;
}
```

Seletor de ID:

Identifica um único elemento através do atributo **id**

```
#cabecalho {
    font-size: 32px;
}
```

ponto (.) : identifica uma classe

hashtag (#) : identifica uma ID

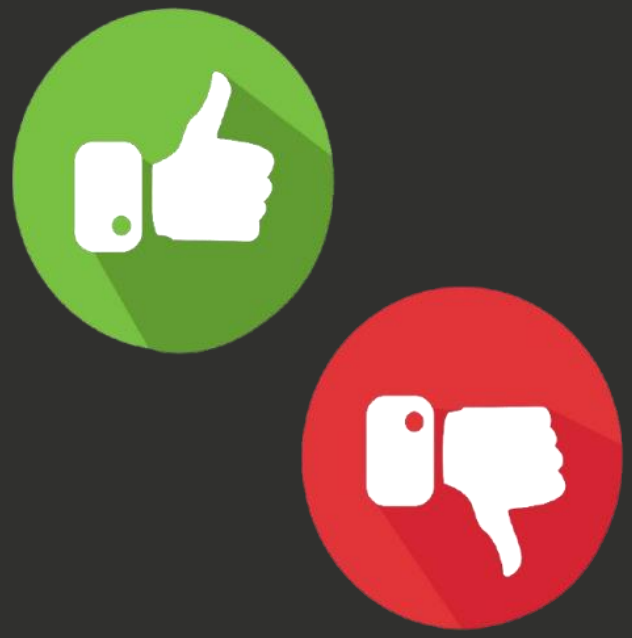
CSS Interno: Análise de Prós e Contras

Vantagens:

- Centralização dos Estilos: Todos os estilos da página em um único local.
- Reutilização na Página: Um seletor aplica estilo a múltiplos elementos.
- Sem Dependências Externas: Tudo contido em um arquivo HTML.

Desvantagens:

- Limitado a Uma Página: Estilos não são compartilhados entre páginas
- Arquivo HTML Maior: Aumenta o tamanho do documento
- Manutenção Difícil: Em sites com múltiplas páginas



Aula 07.3:

CSS externo (external CSS)

CSS Externo: A Solução Profissional

CSS externo mantém os estilos em arquivos separados com extensão **.css**, vinculados ao HTML através da tag **<link>**. Este é o método recomendado para projetos profissionais.

Vinculando o Arquivo CSS:

```
<head>
    <link rel="stylesheet" href="style.css">
</head>
```

Organização Típica:

- **styles.css** – Estilos gerais
- **layout.css** – Estrutura da página
- **components.css** – Componentes reutilizáveis

CSS Externo: Exemplo Completo

Arquivo HTML:

```html
<!DOCTYPE html>
<html>

<head>
    <link rel="stylesheet"
    href="style.css">
</head>

<body>
    <h1 class="titulo">
        Olá Mundo
    </h1>
</body>

</html>
```

Arquivo “style.css:”

```css
.titulo {
    color: #b18ce4;
    font-family: Montserrat;
    font-size: 32px;
    text-align: left;
}

body {
    background-color: #101010;
}
```

Resultado:

O navegador carrega o arquivo CSS e aplica os estilos definidos aos elementos correspondentes no HTML, criando uma página visualmente estilizada.

Olá, mundo!

Por Que CSS Externo é a Melhor Escolha?

Reutilização Total:

Um arquivo CSS serve centenas de páginas HTML, garantindo consistência visual em todo o site.

Manutenção Simplificada:

Alterações em um único arquivo CSS atualizam automaticamente todas as páginas vinculadas.

Performance Otimizada:

Navegadores armazenam em cache arquivos CSS, reduzindo tempo de carregamento em visitas subsequentes.

Código Mais Limpo:

HTML focado em estrutura e conteúdo, CSS focado em apresentação, separação perfeita de responsabilidades

Aula 07.4:

Seletores avançados

Agrupando e Combinando Seletores

O CSS oferece recursos poderosos para **aplicar estilos a múltiplos elementos simultaneamente** ou criar seleções mais específicas através de **combinações**.

Agrupamento com Vírgula

Aplica o mesmo estilo a múltiplos seletores

```
h1, h2, h3 {
font-family: Montserrat;
color: #8CC9E4;
}
```

Seletor Descendente (espaço)

Seleciona elementos dentro de outros

```
article p {
  font-size: 18px;
  line-height: 1.6;
}
```

Seletor de Filho Direto (>)

Seleciona apenas filhos imediatos

```
nav > ul {
list-style: none;
margin: 0;
}
```

Seletores de Irmãos

O CSS oferece recursos poderosos para **aplicar estilos a múltiplos elementos simultaneamente** ou criar seleções mais específicas através de **combinações**.

Irmão Adjacente (+)

Seleciona o elemento imediatamente seguinte

```
h2 + p {
 font-weight: bold;
  margin-top: 0;
}
```

*Neste exemplo, apenas o primeiro parágrafo após cada **h2** será estilizado.*

Dica Prática: Seletores de irmãos são extremamente úteis para criar espaçamentos inteligentes e estilos contextuais sem adicionar classes extras ao HTML.

Irmãos Gerais (~)

```
h2 ~ p {
 color: #FFFFFF;
}
```

*Aplica estilo a todos os parágrafos que seguem um **h2**, não apenas o primeiro.*

Seletores de atributos

Selecionam elementos baseados em seus atributos.

```
input[type="text"] {
    border: 2px solid #8CC9E4;
}

a[href^="https"] {
    color: green;
}
```

E-mail:

Clique aqui para acessar o site

Operador	Significado	Exemplo
^=	começa com	[href^="https"]
$=	termina com	[href$=".pdf"]
***=**	contém	[href*="login"]
=	exatamente igual	[type="checkbox"]
~=	contém palavra inteira	[class~="ativo"]

Aula 07.5:

Pseudo-classes e Pseudo-elementos

Pseudo-classes no CSS

Pseudoclasses em CSS são palavras-chave adicionadas a **seletores**, precedidas por dois pontos (**:**), que permitem estilizar elementos com base em seu estado atual ou posição na estrutura do documento, sem alterar o HTML.

Elas reagem a interações do usuário (como passar o mouse) ou condições estruturais (como o primeiro item de uma lista).

CSS Pseudo-Classes

`:hover`	when an element is hovered when an element is focused
`:focus`	when an element is being activated
`:active`	if a link has been visited if a link has been visited
`:first-child`	if an element is the first child if an element is the last child
`:last-child`	if an element is the nth child if an element is the nth
`:nth-child(2)`	if an element is the nth child if an element is
`:nth-last-child(2)`	if an element is the nth child from the end
`:first-of-type`	if an element is the first of its type
`:not(selector)`	if an element does not match a selector

Pseudo-classes

Pseudo-classes Comuns

Estilos baseados em estado ou posição.

```
a:hover {
    color: #8CC9E4;
    text-decoration: underline;
}


li:first-child {
    font-weight: bold;
}
```

:hover : ativado ao passar o mouse sobre o elemento.

:first-child : primeiro filho do seu elemento pai.

Pseudo-classes Estruturais

Seleção baseada em posição na árvore DOM.

```
tr:nth-child(odd) {
    background: #f0f0f0;
}


p:last-of-type {
    margin-bottom: 0;
}
```

:nth-child*(odd)* : Seleciona elementos na posição ímpar.

:last-of-type : Seleciona o último elemento daquele tipo dentro do elemento pai.

Pseudo-elementos: Criação de Conteúdo Virtual

Pseudo-elementos permitem estilizar partes específicas de elementos ou inserir conteúdo decorativo sem modificar o HTML. Eles usam a sintaxe de dois dois-pontos **(::)**.

::before e ::after

```
h2::before {
    content: "→ ";
    color: #8CC9E4;
    font-weight: bold;
}
```

→ Título

```
blockquote::before {
    content: ' " ';
    font-size: 2em;
}


blockquote::after {
    content: ' " ';
    font-size: 2em;
}
```

" Texto da citação... "

Outros Pseudo-elementos:

::first-line – Primeira linha de texto.

::first-letter – Primeira letra (*usado em "drop caps" : Estilo em que a primeira letra do texto se destaca*).

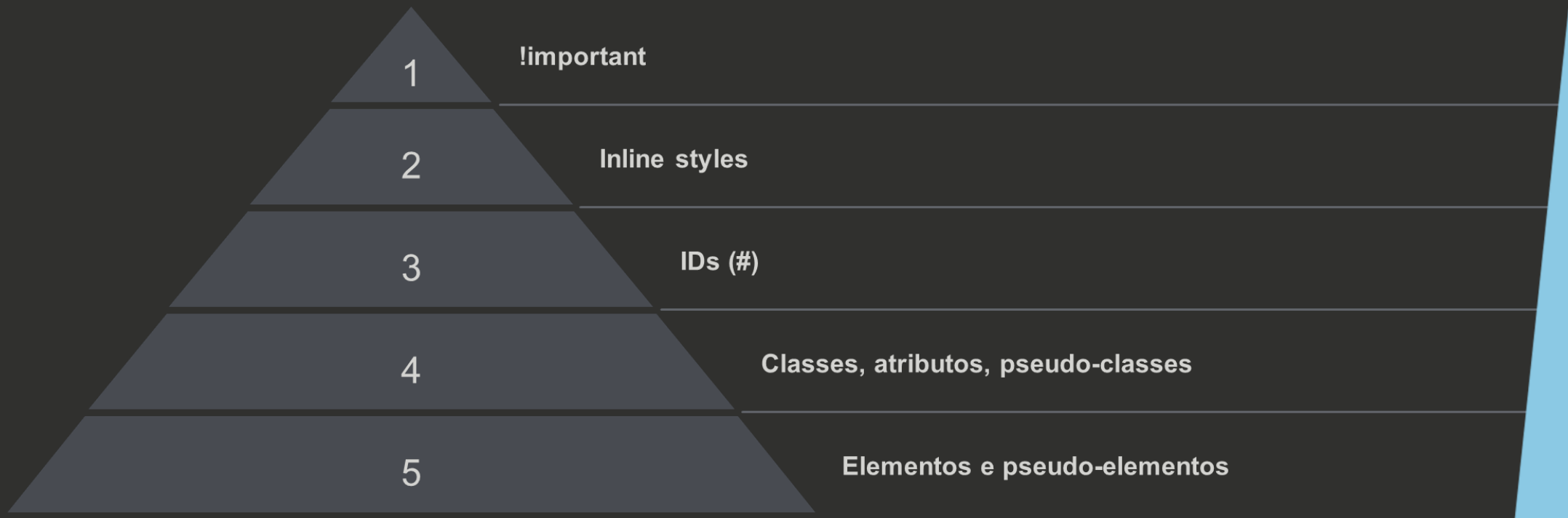
::selection – Texto selecionado pelo usuário.

Aula 07.6:

Especificidade

Especificidade

Quando múltiplas regras CSS se aplicam ao mesmo elemento, o navegador usa a especificidade para determinar qual estilo prevalece. Entender essa hierarquia é fundamental para controlar seus estilos efetivamente.



Quanto mais alto na pirâmide, maior a especificidade. Em caso de empate, a última regra definida no código vence.

O Poder (e Perigo) do !important

Como Funciona:

A declaração **!important** sobrescreve todas as outras regras, independentemente da especificidade:

```
p {
    color: blue !important;
}
```

⚠️ **Armadilhas Comuns:**

- Uso excessivo torna CSS impossível de manter
- Cria "guerras" de especificidade no código
- Dificulta debug e sobrescrita futura
- Indica geralmente má organização do CSS

Use !important como último recurso, nunca como solução padrão.

Quando Usar:

- Sobrescrever estilos de bibliotecas terceiras.
- Corrigir problemas urgentes temporariamente.
- Estilos utilitários que devem sempre prevalecer.

Ferramentas do Desenvolvedor: Inspeção de CSS

Todos os navegadores modernos incluem ferramentas de desenvolvedor (**DevTools**) poderosas para inspecionar, editar e debugar CSS em tempo real. Essas ferramentas são essenciais para qualquer desenvolvedor web.

Como utilizar?

1° - Abrir DevTools

Pressione F12 ou clique com botão direito em qualquer elemento e selecione "**Inspecionar**".

2° - Selecionar Elemento

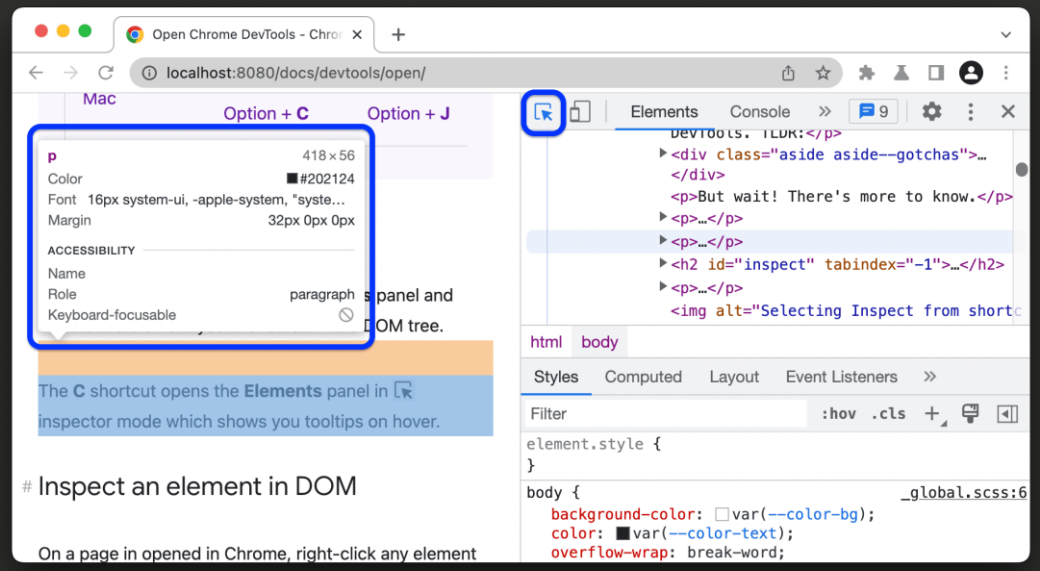
Use a ferramenta de seleção ou navegue pela árvore HTML para encontrar o elemento desejado.

3° - Visualizar Estilos

Examine todos os estilos aplicados, sua origem e especificidade na aba "Styles".

4° - Experimentar Mudanças

Edite propriedades CSS diretamente no painel para testar alterações instantaneamente.



Boas Práticas e Armadilhas Comuns

✓ Melhores Práticas:

Use CSS externo: Mantenha separação entre estrutura e apresentação

Nomes descritivos: Use classes e IDs que descrevam função, não aparência

Organize o código: Agrupe seletores relacionados e comente seções

Evite especificidade alta: Prefira classes a IDs para maior flexibilidade

✗ Erros Comuns:

Abuso de !importante: Cria código difícil de manter e sobrescrever.

Seletores muito específicos: Dificulta reutilização e manutenção futura.

Inline styles em produção: Mistura apresentação com estrutura.

Falta de comentários: Código sem documentação é difícil de entender depois.

Exercício de fixação - Aula 07:

https://forms.office.com/r/FuhsVXsveK

Exercício prático:

https://abre.ai/ouET

Desenvolvimento Web 1

Prof. Diego Max